CyberSNova

Internship Project Report

Name:	Om Vyankat Chavan
Email:	Omchavan416@gmail.com
Reference No:	CSN-OC-OL-26294
Registration No.	CSN-OC-2026-294
Domain:	Cyber Security
Project Level:	Beginner

S.No.	Project Description	Page No.
1	Option 3: OverTheWire Bandit Challenge. Step-by-step walkthrough of Bandit Levels 0 to 20 (SSH wargame), explaining the objective, commands used, reasoning and the password obtained for the next level, with screenshots, findings, summary and recommendations.	2

OVERTHEWIRE BANDIT WARGAME REPORT

Option 3: Bandit Levels 0 to 20, a complete learning journey

Student Name	Om Vyankat Chavan
Roll Number	093019
Subject	Cyber Security / Linux and Network Fundamentals
Platform	OverTheWire Bandit (https://overthewire.org/wargames/bandit/)
Tools Used	OpenSSH client, Linux command line (coreutils), nmap, openssl s_client, netcat, xxd, gzip, bzip2, tar, nano
Test Environment	Kali Linux 2025.4 (VMware), connecting to bandit.labs.overthewire.org on port 2220
Date of Completion	25 October 2026

1. Objective

Bandit is the first wargame on OverTheWire and is aimed at people who are new to the Linux command line and to security thinking. Every level gives you one small puzzle. If you solve it, you get a password, and that password lets you log in as the next user (bandit1, bandit2 and so on) through SSH.

The aim of this practical was to work through Levels 0 to 20 myself, understand why each solution works instead of just copying a command, and then document the whole journey. For every level this report explains the objective, the commands I used, the reasoning behind them, and the password I obtained for the next level. I have also included the mistakes I made on the way, because most of what I learned came from fixing them.

2. Platform and Tools

Platform: OverTheWire Bandit is a free, legal practice server. All levels are reached with one SSH command pattern: `ssh banditN@bandit.labs.overthewire.org -p 2220`.

- **ssh / ssh -i:** to log in with a password or with a private key.
- **Core Linux utilities:** ls, cat, file, find, du, grep, sort, uniq, strings, base64, tr, diff, head, chmod, cp, mv.
- **Data and archive tools:** xxd (reverse a hexdump), gzip, bzip2, tar.
- **Network tools:** nc (netcat), nmap for port and service discovery, and openssl s_client for TLS connections.
- **nano:** to paste and save a private key on my own Kali machine.

3. Scope and Ethical Note

Bandit is designed to be attacked by learners. I only connected to the official OverTheWire Bandit server, only used the accounts the game provides, and only did what each level asks for. From Level 16 onward I used nmap, but only against localhost from inside the Bandit machine, as the level instructs, and never against any other system. I also read the OverTheWire rules, which the first level points to, and followed them.

4. Methodology

- Read the goal on the official Bandit level page, then log in as that level's user.
- Look around first: ls, ls -la, file and pwd tell you what you are dealing with before you try anything clever.
- Pick the simplest tool that fits the clue (for example grep for "find one word", sort and uniq for "find the unique line").
- Copy the password carefully, exit, and log in as the next user. A wrong paste is the most common reason for "Permission denied".
- Take a screenshot of the working command and its output, and note down any errors and how I fixed them.

5. Level-by-Level Walkthrough

Each section below follows the same pattern: Objective, Commands Used, Reasoning, and the password for the next level. Passwords are written exactly as they appeared in my terminal screenshots.

Level 0 → Level 1

Objective

Log into the game for the first time using SSH on port 2220 with the username bandit0 and the password bandit0, then find the password for Level 1 stored in a file called readme in the home directory.

Commands Used

```
ssh bandit0@bandit.labs.overthewire.org -p 2220
ls
cat readme
```

Reasoning

Bandit does not use the normal SSH port 22, so the -p 2220 option is essential. After logging in, ls showed a single file called readme. cat prints a file to the screen, and the file told me to read the OverTheWire rules and gave the password.



Figure 1: First SSH login to the Bandit server as bandit0 on port 2220.

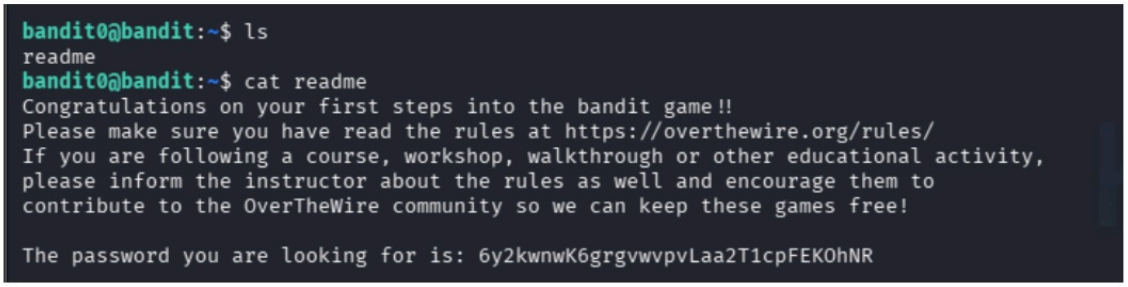


Figure 2: ls and cat readme revealing the password for Level 1.

> **Password for Level 1: `6y2kwnwK6grgvwvpvLaa2T1cpFEKOhNR`**

Level 1 → Level 2

Objective

The password is stored in a file simply called "-" (a single dash) in the home directory.

Commands Used

```
ls
cat ./-
```

Reasoning

If I type cat - the shell does not look for a file; most Linux tools treat a lone dash as "read from standard input" and the terminal just waits. Writing ./- gives the path of the file in the current directory, so the program treats it as a real filename. This was my first lesson that special characters in filenames need care.



Figure 3: Reading the file named "-" using the ./ prefix.

> **Password for Level 2: `PK8fYLZg2hnHSz83plBL1iEPKdD3QToB`**

Level 2 → Level 3

Objective

The password is in a file called "--spaces in this filename--" in the home directory.

Commands Used

```
ls
cat < "--spaces in this filename--"
```

Reasoning

This filename has two problems at once: spaces, which split a command into separate words, and a leading double dash, which cat would mistake for an option. Putting the name inside quotes solves the spaces. Using input redirection with < solves the dashes, because the shell opens the file itself and hands the content to cat as standard input, so cat never sees the name. Using ./ with escaped spaces would also work.

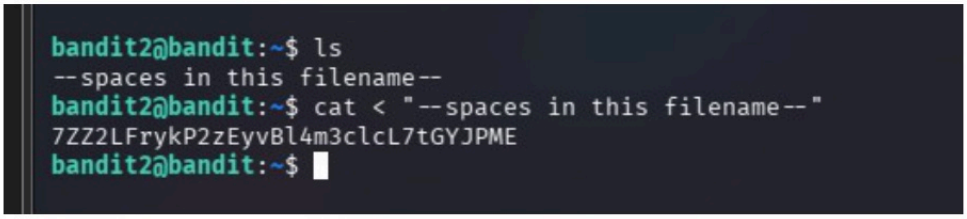


Figure 4: Reading the file with spaces and dashes using quotes and input redirection.

> **Password for Level 3: `7ZZ2LFrykP2zEyvBl4m3clcL7tGYJPME`**

Level 3 → Level 4

Objective

The password is stored in a hidden file inside the directory called inhere.

Commands Used

```
cd inhere
ls
ls -a
cat .Hiding-From-You
```

Reasoning

A plain ls showed nothing inside inhere, which is normal for a hidden file. On Linux, any file whose name begins with a dot is hidden from the default listing. ls -a ("all") reveals dotfiles, and I could then read the hidden file with cat. This is the same idea as files like .bashrc and .ssh in a real home directory.

The password from this file was used to log in as bandit4.

Level 4 → Level 5

Objective

The inhere directory contains ten files named -file00 to -file09. Only one of them is human-readable, and it holds the password.

Commands Used

```
cd inhere/
ls -la
file ./*
cat ./-file07
```

Reasoning

Reading ten unknown files with cat would dump binary junk into my terminal and could even mess up the display. The file command looks at the contents and reports the type. Nine files were reported as "data" and only -file07 was "ASCII text", so that was the one to read. Because the names start with a dash, I again used the ./ prefix.

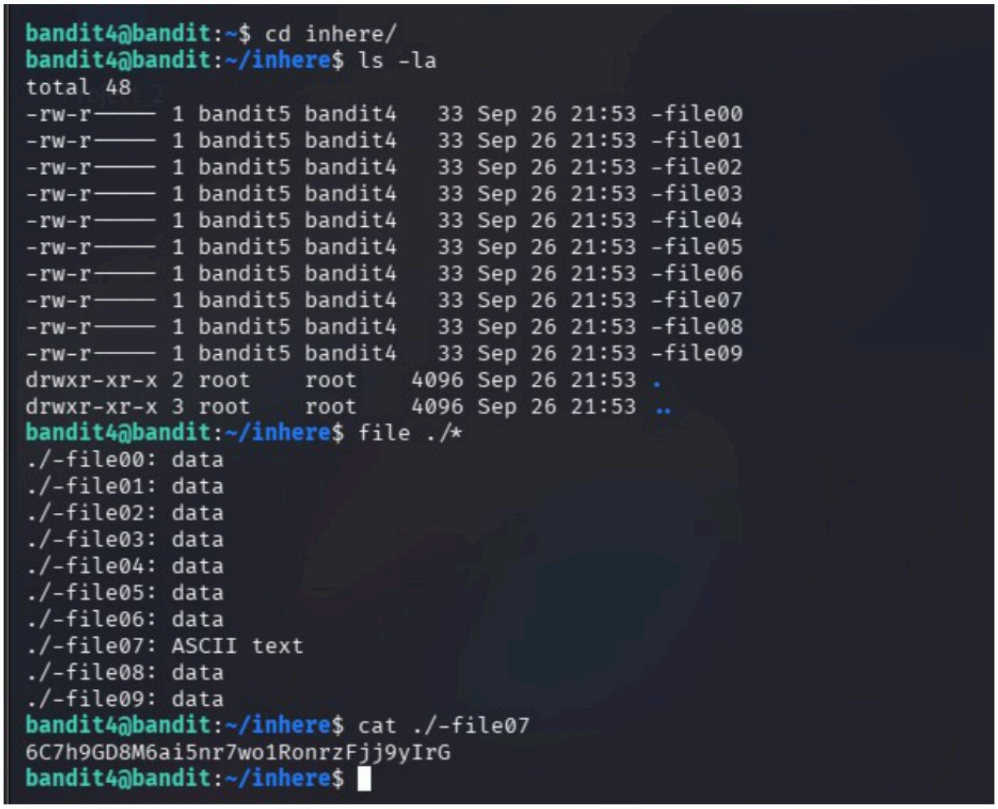


Figure 5: file ./ shows -file07 as the only ASCII text file.*

> **Password for Level 5: `6C7h9GD8M6ai5nr7wo1RonrzFjj9yIrG`**

Level 5 → Level 6

Objective

Inside inhere there are 20 sub-directories. Exactly one file among them is human-readable, exactly 1033 bytes in size, and not executable.

Commands Used

```
cd inhere
ls -la
ls -la maybehere00
file /{.,}* | grep "ASCII text" | grep -v ', whith very long lines'
du -b -a | grep 1033
cat ./maybehere07/.file2
```

Reasoning

I listed the directories first and then looked inside maybehere00 to see what a typical folder contains: normal files, hidden files and files with spaces in the names. Checking 20 folders by hand would be slow, so I tried to filter by file type with file and grep. That attempt printed nothing useful, so I switched to the size clue instead. du -b -a prints every file with its exact size in bytes, and piping that into grep 1033 pointed straight to ./maybehere07/.file2. Its permissions (-rw-r-----) showed it was not executable, which matched the description.

A cleaner one-line alternative would have been `find . -type f -size 1033c ! -executable`*, which combines all three clues in a single command.*

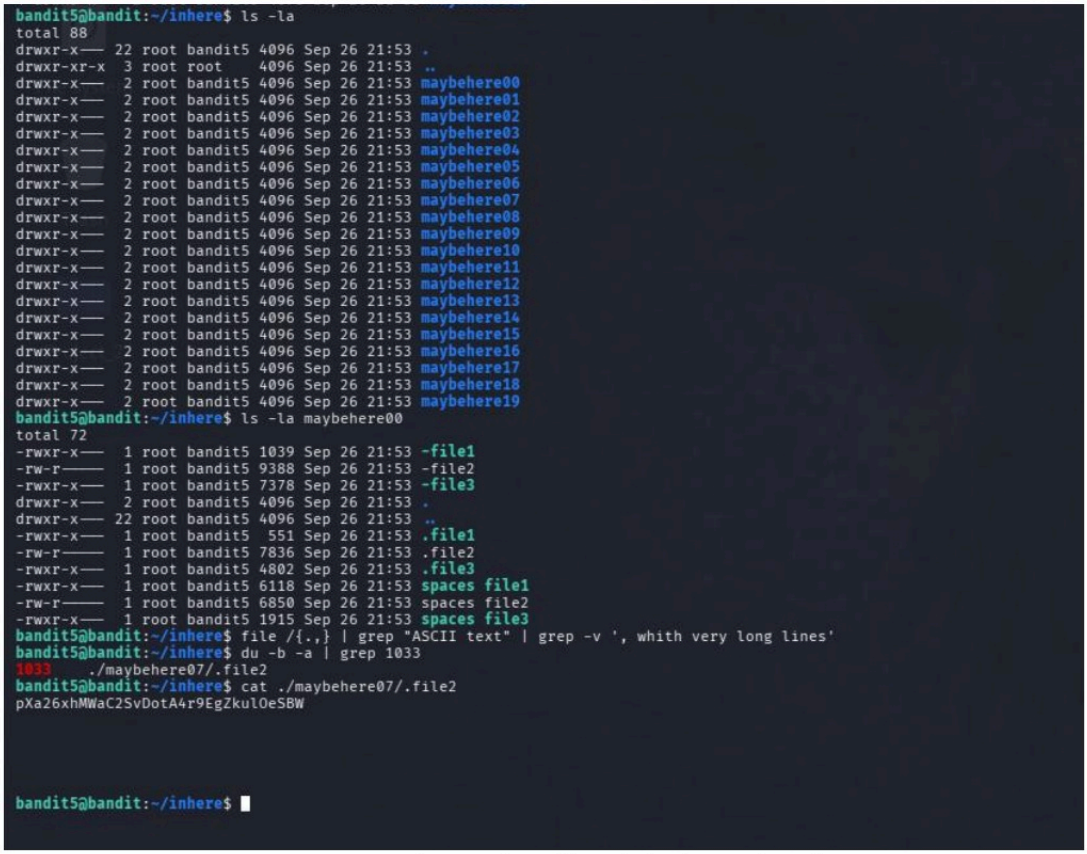


Figure 6: Locating the 1033-byte file with du -b -a and reading it.

> **Password for Level 6: `pXa26xhMWaC2SvDotA4r9EgZkulOeSBW`**

Level 6 → Level 7

Objective

The password is somewhere on the server. The file is owned by user bandit7, owned by group bandit6, and is 33 bytes in size.

Commands Used

```
cd ..
ls
cd bandit7
ls -la
cat data.txt
find / -type f -user bandit7 -group bandit6 -size 33c 2>/dev/null
cat /var/lib/dpkg/info/bandit7.password
```

Reasoning

I first looked in /home to see how the users are laid out. The data.txt in bandit7's directory gave "Permission denied", so that was not the file I needed. The clue lists three properties, and find can test all three at once: -user, -group and -size (the c means bytes). Searching from / covers the whole system, which produces a lot of "Permission denied" errors, so I added 2>/dev/null to discard error messages and keep only the real result. It returned a single path, /var/lib/dpkg/info/bandit7.password.

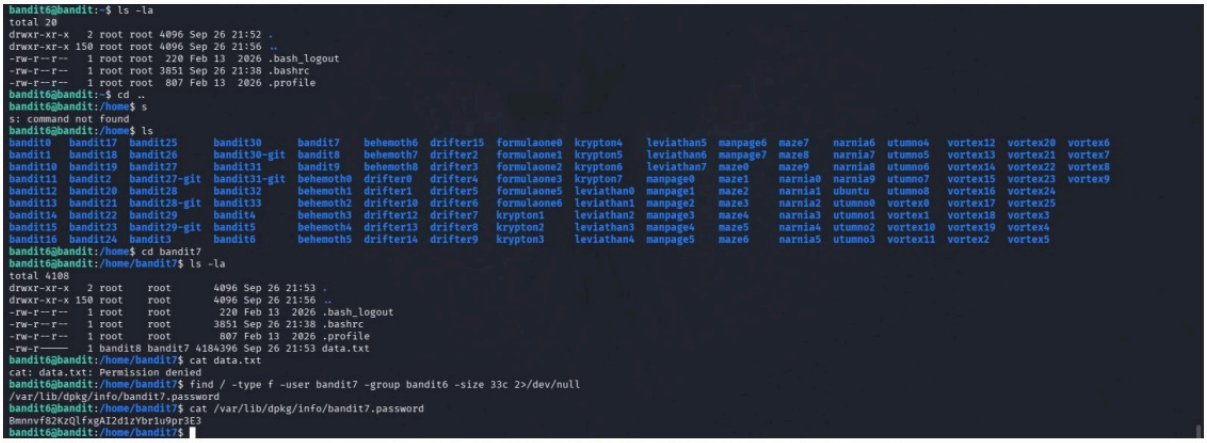


Figure 7: Using find with user, group and size filters to locate the password file.

> **Password for Level 7: `Bmnnvf82KzQlfxgAI2d1zYbr1u9pr3E3`**

Level 7 → Level 8

Objective

The password is in data.txt next to the word "millionth".

Commands Used

```
head -n 10 data.txt
grep millionth data.txt
```

Reasoning

head -n 10 let me peek at the file format safely: each line is a word followed by a long random string. The file is far too big to read by eye, so grep prints only the line that contains the keyword. That is the entire level, but it taught me how powerful grep is for big text files.

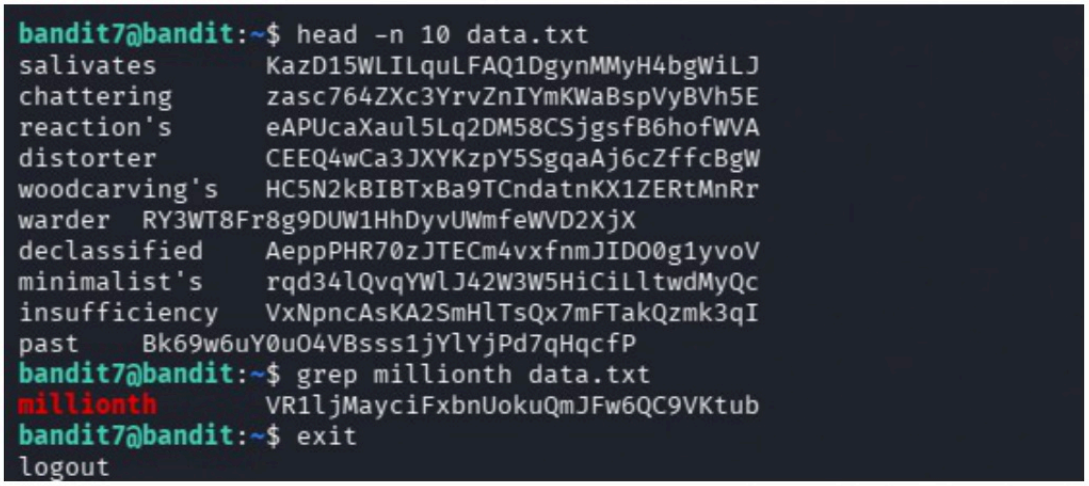


Figure 8: grep millionth data.txt returns the line with the password.

> **Password for Level 8: `VR1ljMayciFxbnUokuQmJFw6QC9VKtub`**

Level 8 → Level 9

Objective

The password is the only line in data.txt that appears exactly once; every other line is repeated.

Commands Used

```
head -n 10 data.txt
```

```
cat data.txt | sort | uniq -u
```

Reasoning

uniq only compares neighbouring lines, so duplicates must sit next to each other before it can work. That is why sort comes first. After sorting, uniq -u prints only the lines that are not repeated, which leaves exactly the password.

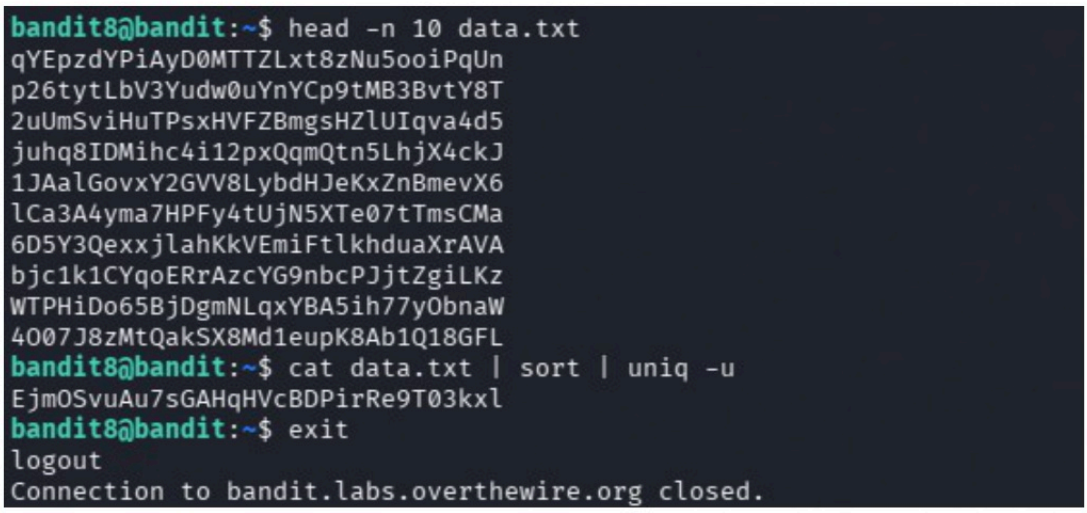


Figure 9: sort | uniq -u isolating the single unique line.

> **Password for Level 9: `EjmOSvuAu7sGAHqHVcBDPirRe9T03kxl`**

Level 9 → Level 10

Objective

The password is one of the few human-readable strings inside data.txt, and it is preceded by several "=" characters.

Commands Used

```
ls
file data.txt
strings data.txt | grep -E "=+"
```

Reasoning

file reported plain "data", which means a normal cat would show mostly garbage. The strings command extracts only the printable character sequences from binary content. Piping that into grep -E "=+" (one or more equals signs) kept just the interesting lines, and the last one held the password.

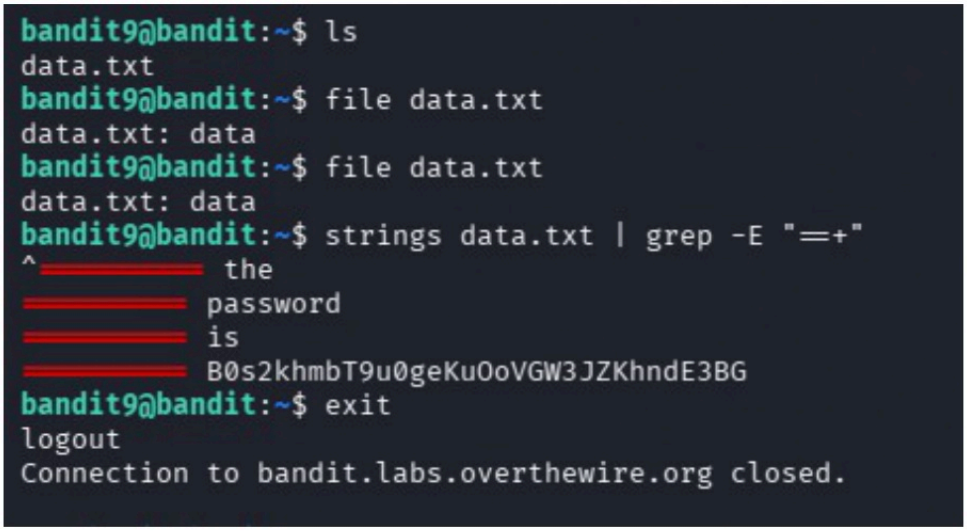


Figure 10: strings piped into grep shows the sentence containing the password.

> **Password for Level 10: `B0s2khmbT9u0geKuOoVGW3JZKhndE3BG`**

Level 10 → Level 11

Objective

The password is in data.txt, which contains Base64 encoded data.

Commands Used

```
ls -la
cat data.txt
cat data.txt | base64 --decode
```

Reasoning

The content ended with "==", which is the typical padding of Base64 text. I made a small typo at first (data,txt instead of data.txt) and got "No such file or directory", then ran the correct command. Decoding with base64 --decode printed the sentence containing the password. An important point for security: Base64 is only an encoding to make binary data printable, not encryption, and anyone can reverse it.

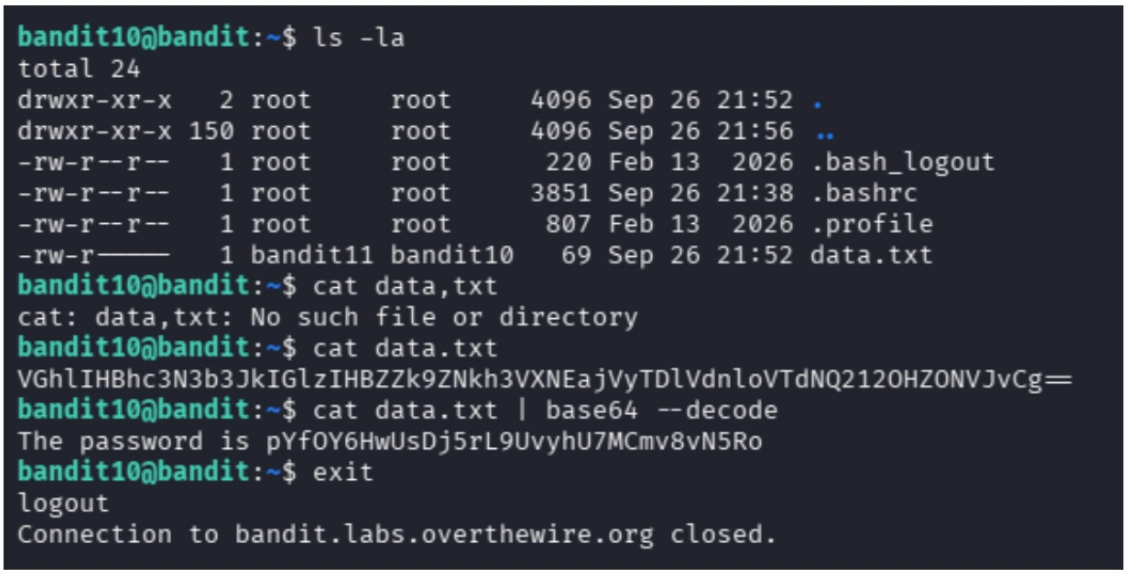


Figure 11: Decoding the Base64 text with base64 --decode.

> **Password for Level 11: `pYfOY6HwUsDj5rL9UvyhU7MCmv8vN5Ro`**

Level 11 → Level 12

Objective

The password is in data.txt, where all lowercase and uppercase letters have been rotated by 13 positions (ROT13).

Commands Used

```
ls
cat data.txt | tr 'A-Za-z' 'N-ZA-Mn-za-m'
```

Reasoning

ROT13 is a simple substitution cipher in which A becomes N, B becomes O and so on. The tr command translates characters from one set to another, so mapping A-Z to N-ZA-M (and a-z to n-za-m) reverses the shift. Applying ROT13 twice returns the original text, so the same command encodes and decodes. This is a good reminder that classical ciphers like this offer no real protection.

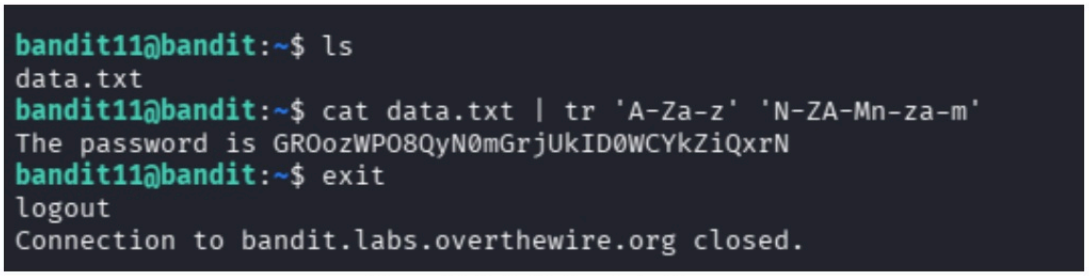


Figure 12: ROT13 decoded with tr.

> **Password for Level 12: `GROozWPO8QyN0mGrjUkID0WCYkZiQxrN`**

Level 12 → Level 13

Objective

The password is in data.txt, which is a hexdump of a file that has been compressed many times over. The level asks you to work in a temporary directory because the home directory is read-only.

Commands Used

```
mktemp -d                       (creates /tmp/tmp.fkZztLbgCx)
cd /tmp/tmp.fkZztLbgCx
cp ~/data.txt .  &&  mv data.txt hexdump_data
xxd -r hexdump_data compressed_data
xxd compressed_data | head       (identify the compression type from the
first bytes)
mv compressed_data compressed_data.gz   &&  gzip -d compressed_data.gz
mv compressed_data compressed_data.bz2  &&  bzip2 -d compressed_data.bz2
mv compressed_data compressed_data.gz   &&  gzip -d compressed_data.gz
mv compressed_data compressed_data.tar  &&  tar -xf compressed_data.tar
tar -xf data5.bin   ->   bzip2 -d data6.bin   ->   tar -xf data6.bin.out
mv data8.bin data8.gz  &&  gzip -d data8.gz
cat data8
```

Reasoning

I created a private working folder with mktemp -d so that I could create files freely. xxd -r reverses a hexdump back into the original binary. After that the work is a loop: look at the first bytes of the file, decide what it is, rename it with the right extension, and decompress it. A file starting with 1f 8b is gzip, one starting with "BZh" (42 5a 68) is bzip2, and a tar archive has the "ustar" marker and the inner file

name in the header. gzip and bzip2 refuse to work unless the extension matches, which is why I kept renaming files with mv.

I made several slips here. I ran gzip -d twice on the same file and got "No such file or directory", because the first run had already removed the .gz. bzip2 also complained "Can't guess original name" when the file had no recognised extension. Each time, the fix was simply to re-check what the current file was with xxd or file and rename it correctly. After about eight layers the last file, data8, contained the password in plain text.

```
bandit12@bandit:~$ cd /tmp
bandit12@bandit:/tmp$  mktemp -d
/tmp/tmp.fkZztLbgCx
bandit12@bandit:/tmp$ cd /tmp/tmp.5F9tJgRwts
-bash: cd: /tmp/tmp.5F9tJgRwts: No such file or directory
bandit12@bandit:/tmp$ cd /tmp/tmp.5F9tJgRwts
-bash: cd: /tmp/tmp.5F9tJgRwts: No such file or directory
bandit12@bandit:/tmp$ ls
ls: cannot open directory '.': Permission denied
bandit12@bandit:/tmp$ xs
xs: command not found
bandit12@bandit:/tmp$ cd
bandit12@bandit:~$ cd
bandit12@bandit:~$ cd /tmp
bandit12@bandit:/tmp$ cd /tmp/tmp.fkZztLbgCx
bandit12@bandit:/tmp/tmp.fkZztLbgCx$ cp ~/data.txt .
bandit12@bandit:/tmp/tmp.fkZztLbgCx$ ls
data.txt
bandit12@bandit:/tmp/tmp.fkZztLbgCx$ mv data.txt hexdump_data
bandit12@bandit:/tmp/tmp.fkZztLbgCx$ ls
hexdump_data
bandit12@bandit:/tmp/tmp.fkZztLbgCx$ ls
hexdump_data
bandit12@bandit:/tmp/tmp.fkZztLbgCx$ cat hexdump_data | head
00000000: 1f8b 0808 b83e b86a 0203 6461 7461 322e  .....>.j..data2.
00000010: 6269 6e00 0146 02b9 fd42 5a68 3931 4159  bin..F...BZh91AY
00000020: 2653 59fc 8b90 bd00 0019 ffff dde3 ddf7  &SY.............
00000030: feaf 7f1f ffdf fdeb bfff f6fb bf7f 8b7f  ................
00000040: 9ffb b7e8 ffd7 afe6 d7ff fdbd b001 3b56  ..............;V
00000050: a100 6800 1a1a 0068 34f5 0000 6869 a1a1  ..h....h4...hi..
00000060: a00c 9a06 8000 0d00 0000 0680 001a 00d1  ................
00000070: 91a3 4064 1e48 189e 9911 0000 00d0 1a0f  ..@d.H..........
00000080: 51a0 0d0d 0d34 0686 81a0 680d 0000 0190  Q....4....h.....
00000090: 34d0 184d 069a 068d 3d41 b503 40d3 6a00  4..M....=A..@.j.
bandit12@bandit:/tmp/tmp.fkZztLbgCx$ xxd -r hexdump_data compressed_data
bandit12@bandit:/tmp/tmp.fkZztLbgCx$ ls
compressed_data  hexdump_data
bandit12@bandit:/tmp/tmp.fkZztLbgCx$ xxd -r hexdump_data compressed_data
bandit12@bandit:/tmp/tmp.fkZztLbgCx$ ls
```

Figure 13: Creating the temporary directory, copying data.txt and reversing the hexdump with xxd -r.

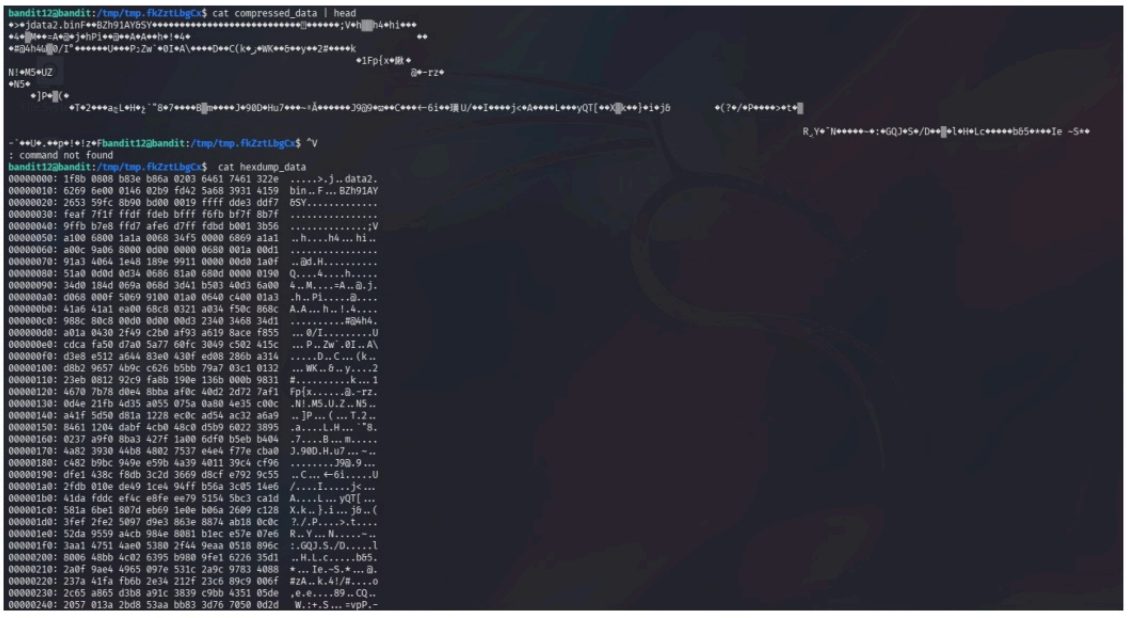


Figure 14: Checking the file header after xxd -r to identify the compression format.



Figure 15: Renaming and decompressing with gzip and bzip2 (including the repeated gzip error).

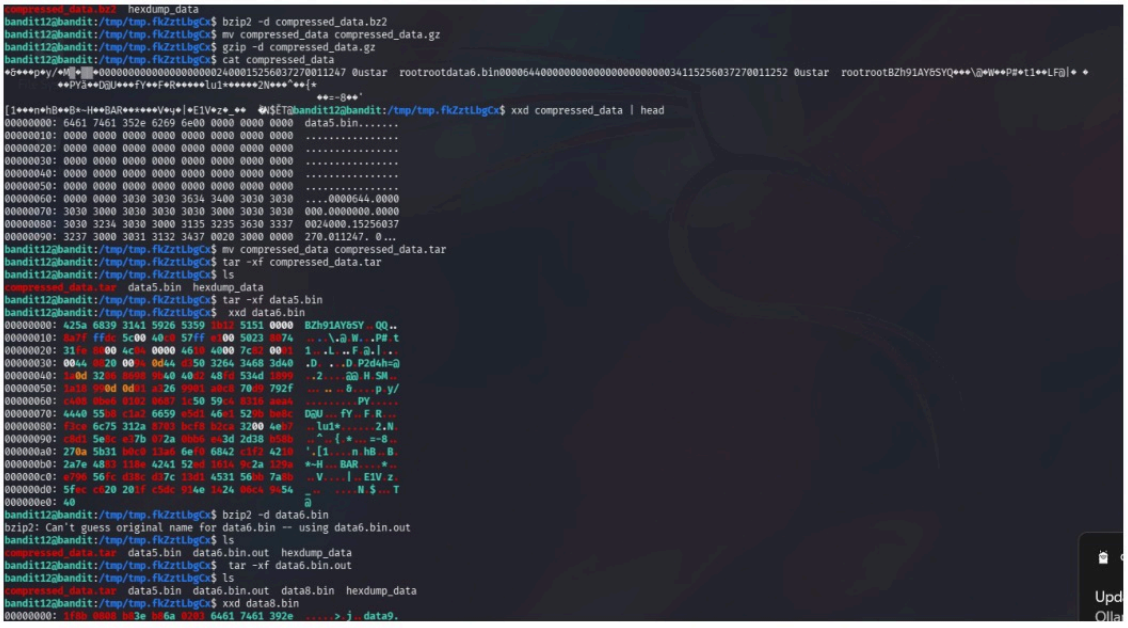


Figure 16: Extracting the nested tar archives (data5.bin, data6.bin) and reaching data8.bin.

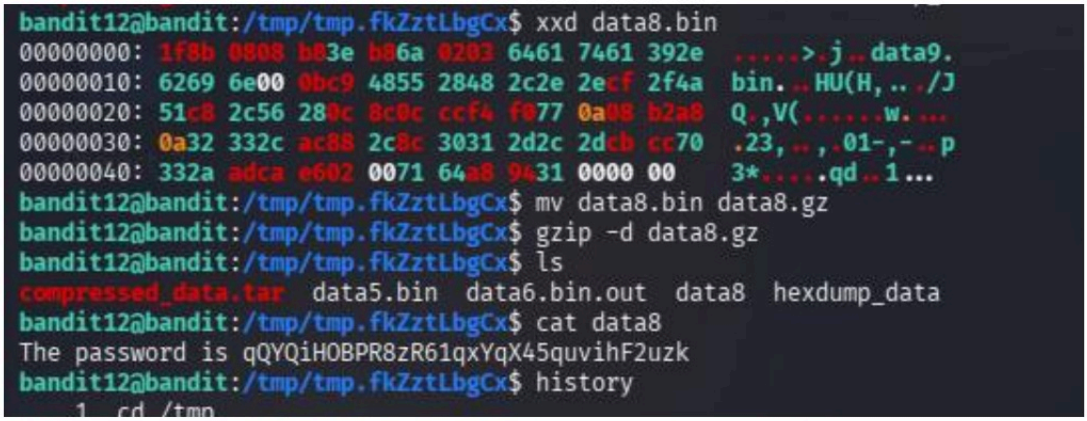


Figure 17: Final decompression of data8.gz and the plain-text password.

> **Password for Level 13: `qQYQiHOBPR8zR61qxYqX45quvihF2uzk`**

Level 13 → Level 14

Objective

Instead of a password, the home directory of bandit13 contains a private SSH key (sshkey.private) that can be used to log in as bandit14. The password for bandit14 can only be read by bandit14, so the key is the way in.

Commands Used

```
ssh bandit13@bandit.labs.overthewire.org -p 2220
cat sshkey.private
# on my Kali machine:
nano bandit14.key        (paste the key)
chmod 600 bandit14.key
ssh -i bandit14.key bandit14@bandit.labs.overthewire.org -p 2220
cat /etc/bandit_pass/bandit14
```

Reasoning

Public-key authentication works like this: the server stores my public key, and I prove who I am by holding the matching private key. Because the key is just text, I printed it with cat. My first idea was to save it to a file on the Bandit server with nano, but nano failed with "Error writing bandit14.key.private: Permission denied", since users cannot write in their home directory on this server. So I pasted the key into a new file on my own Kali machine instead.

The most important step is chmod 600. SSH refuses to use a private key that other users can read ("unprotected private key file"), so the file must be accessible only by its owner. Once the key worked, I was logged in as bandit14 and could read the password file at /etc/bandit_pass/bandit14.



Figure 18: Logging in as bandit13 with the password from Level 12.

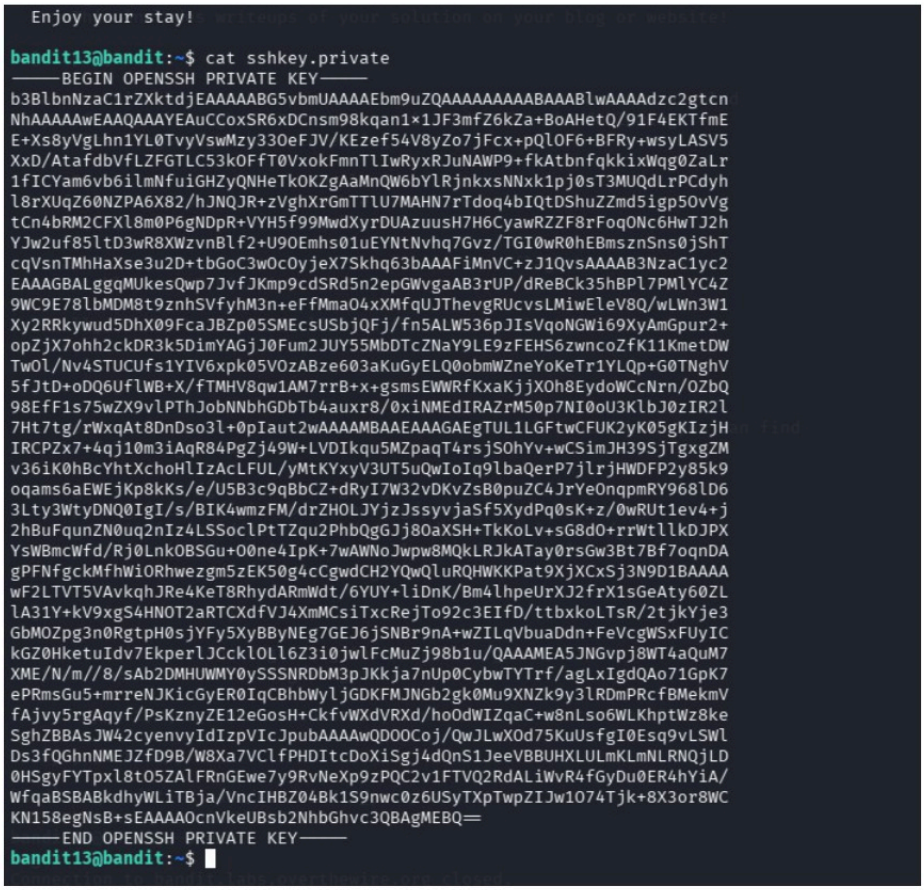


Figure 19: Printing the private key (sshkey.private) with cat.

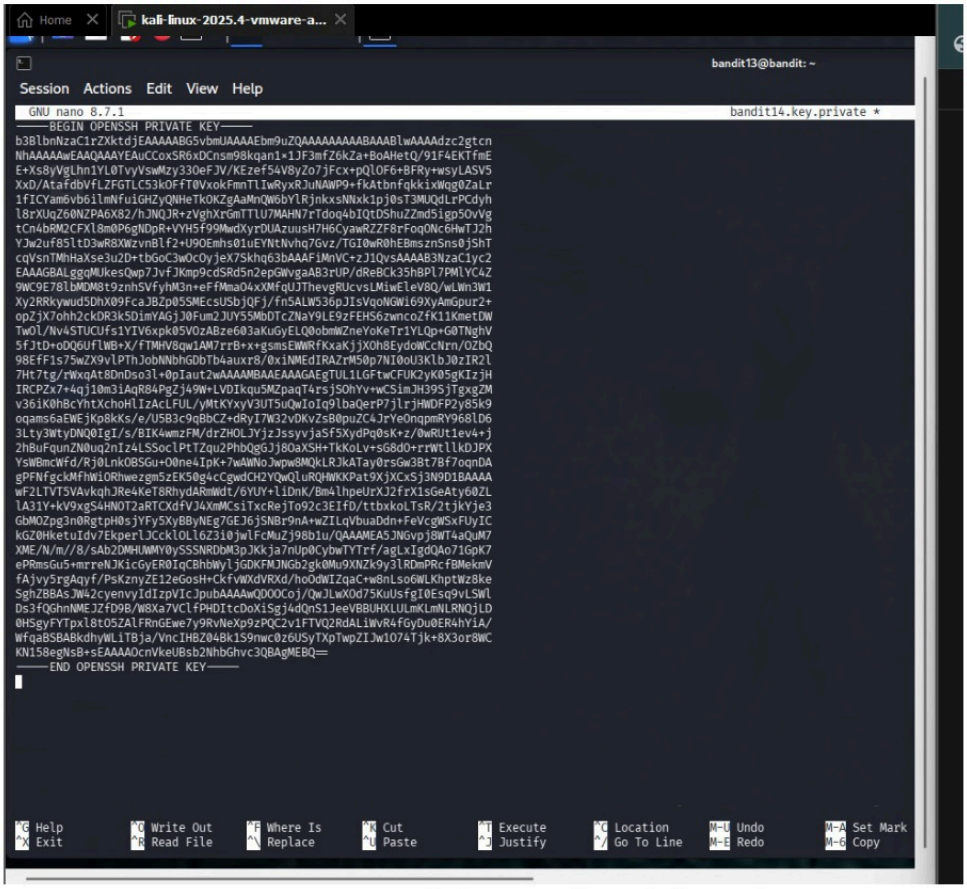


Figure 20: Attempting to paste the key into a file inside the Bandit session.

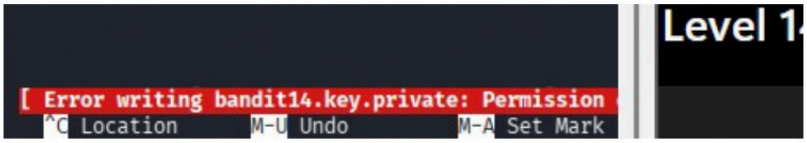


Figure 21: The permission error shown when trying to save the key on the server.

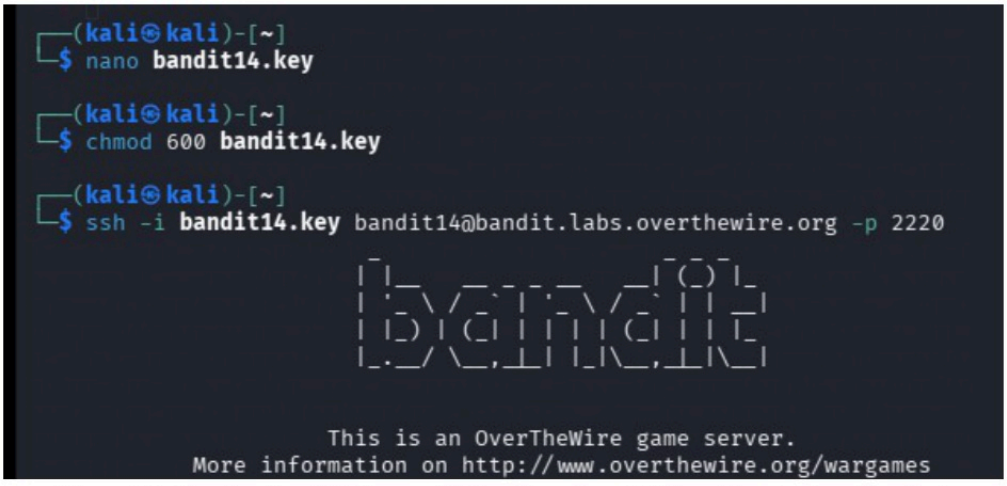


Figure 22: Creating bandit14.key on Kali, setting chmod 600 and logging in with ssh -i.

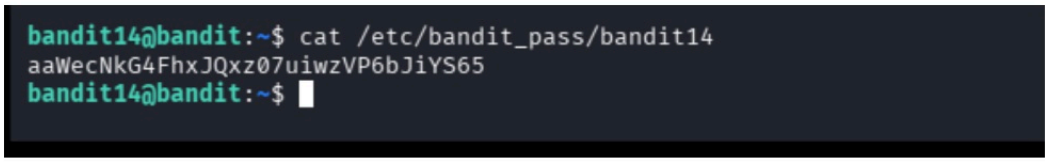


Figure 23: Logged in as bandit14 and reading /etc/bandit_pass/bandit14.

> **Password for bandit14 (read from /etc/bandit_pass/bandit14):**
> **`aaWecNkG4FhxJQxz07uiwzVP6bJiYS65`**

Level 14 → Level 15

Objective

Submit the current level's password to port 30000 on localhost. If it is correct, the service returns the password for the next level.

Commands Used

```
echo <old password> | nc localhost 30000       (wrong, I used an old password
by mistake)
echo aaWecNkG4FhxJQxz07uiwzVP6bJiYS65 | nc localhost 30000
```

Reasoning

nc (netcat) opens a raw TCP connection, and echo ... | nc sends the string and prints the reply. My first attempt used a password left over from an earlier session and the service answered "Wrong! Please enter the correct current password." I also pasted the line once with a stray character, which produced "command not found". After using the password I had just read from /etc/bandit_pass/bandit14, the service replied "Correct!" and printed the next password. This level showed me that services listening on a port can be talked to directly from the terminal.



Figure 24: Wrong attempts followed by the correct password sent to port 30000 with nc.

> **Password for Level 15: `pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7`**

Level 15 → Level 16

Objective

Submit the current password to port 30001 on localhost, but this time the connection must use SSL/TLS encryption.

Commands Used

```
openssl s_client -connect localhost:30001
(paste the Level 15 password when the connection is ready)
```

Reasoning

nc cannot speak TLS, so the service would not understand it. openssl s_client is a general-purpose TLS client: it performs the handshake, shows the certificate chain, and then lets me type like in netcat. The output showed a self-signed certificate (CN=SnakeOil, verify error 18), which is fine for a practice server but would be a red flag on a real website. After the session details, the program waited for input. I pasted the password, the server replied "Correct!" and printed the next password.

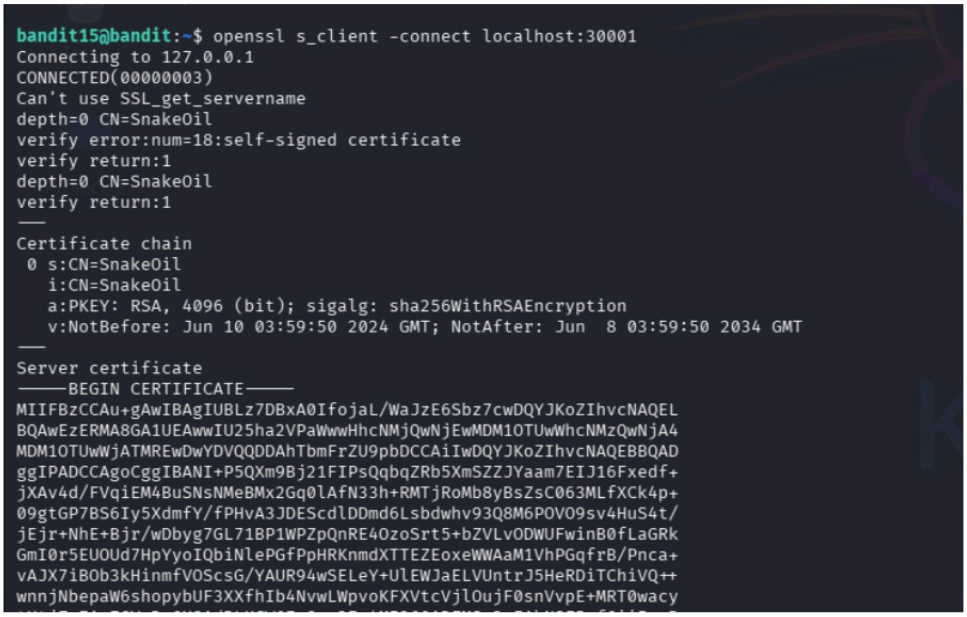


Figure 25: TLS handshake with openssl s_client showing the self-signed SnakeOil certificate.



Figure 26: Session information, then the password submitted and the reply "Correct!".

> **Password for Level 16: `kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V`**

Level 16 → Level 17

Objective

The next credentials are on a port between 31000 and 32000 on localhost. Only one of those ports speaks SSL and gives the answer; the others just echo what you send. The reward is an SSH private key.

Commands Used

```
nmap -sV localhost -p 31000-32000
echo kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V | openssl s_client -connect
localhost:31790 -quiet
# save the key on Kali, then fix and use it:
file ~/bandit17key.private
head -n 2 ~/bandit17key.private
sed '1{/^Correct!$/d;}' ~/bandit17key.private > ~/bandit17_clean.key
chmod 600 ~/bandit17_clean.key
ssh-keygen -y -f ~/bandit17_clean.key
ssh -i ~/bandit17_clean.key bandit17@bandit.labs.overthewire.org -p 2220
```

Reasoning

nmap -sV scans a range of ports and tries to identify the service on each one. Five ports were open: 31046, 31518, 31691, 31790 and 31960. Most were identified as simple "echo" services, while 31790 appeared as ssl/unknown. 31518 also showed ssl/echo, which is a decoy that only repeats what you send. Connecting to 31790 with openssl s_client and the current password returned "Correct!" followed by an RSA private key.

Getting that key to work took me the longest time of all the levels. Everything I tried gave an error, and the screenshots below show the process honestly:

- My first attempts failed because I used the wrong file names (cp sshkey.private ... and chmod on a file that did not exist yet), so I had no real key file.
- After saving the key with nano, SSH printed "Load key ... error in libcrypto" and fell back to asking for a password.
- I tried ssh -i with a different file name and even used sudo -i, which only produced "Identity file not accessible" and "Permission denied".
- The real cause appeared when I ran file and head -n 2 on the key: the first line of my file was the word "Correct!" that the server prints before the key, so the key did not start with -----BEGIN OPENSSH PRIVATE KEY-----.
- I removed that line with sed, saved the result as bandit17_clean.key, applied chmod 600, confirmed it with ssh-keygen -y (it printed a valid public key), and then the login to bandit17 worked.

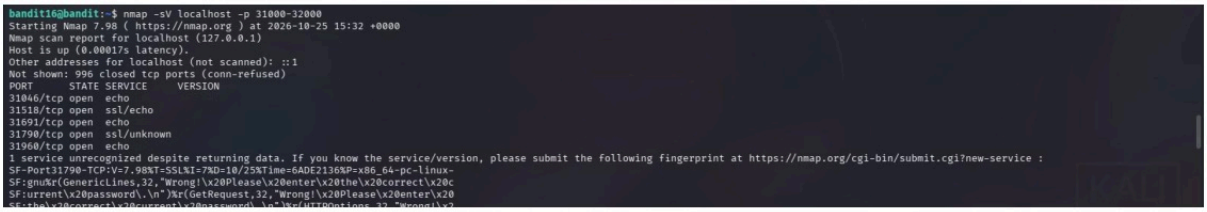


Figure 27: nmap -sV scan of ports 31000-32000 on localhost.

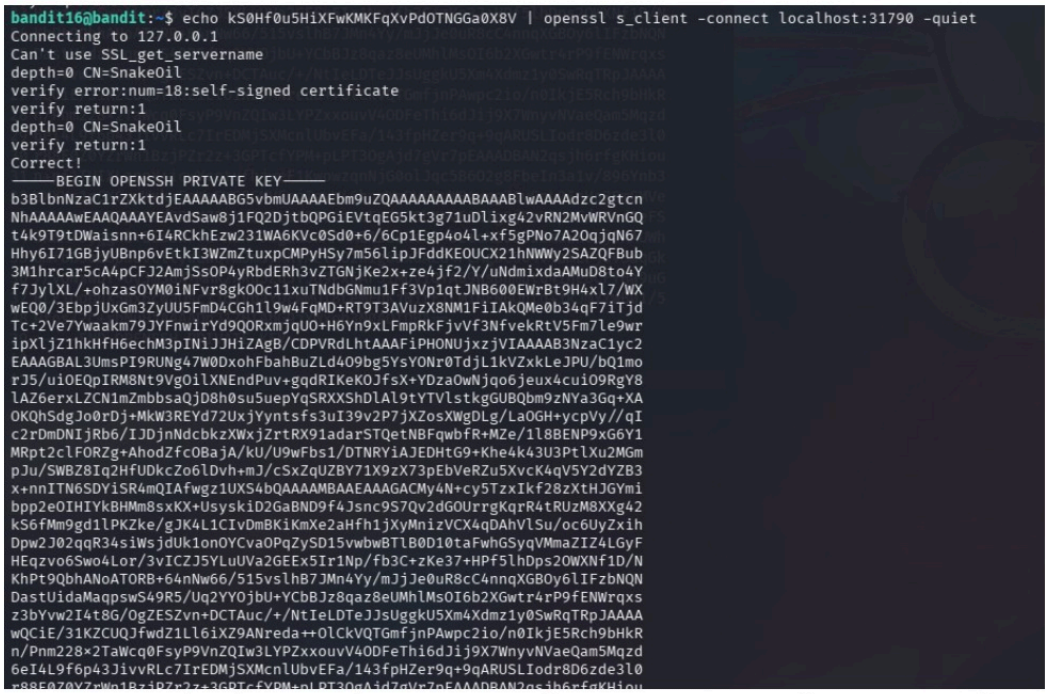


Figure 28: Sending the password to port 31790 and receiving "Correct!" with an RSA private key.

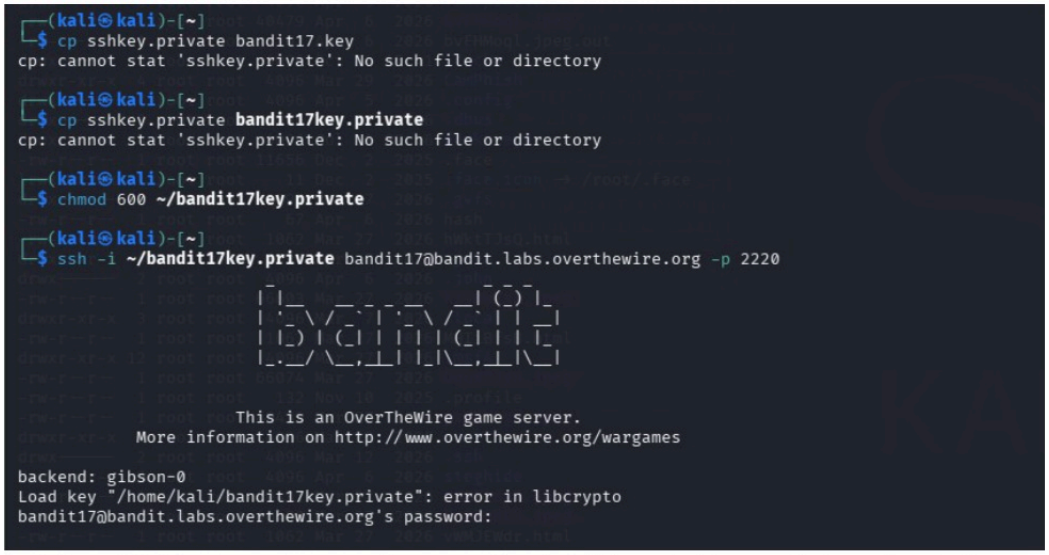


Figure 29: First attempt: wrong file name, chmod and "error in libcrypto" when using the key.

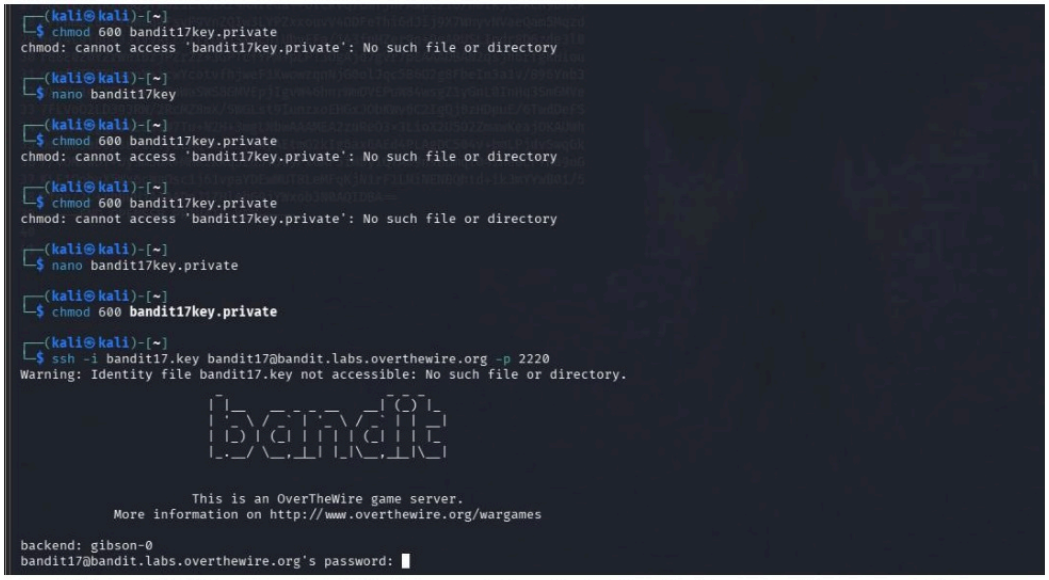


Figure 30: More attempts with nano and chmod; the identity file was still not accessible.

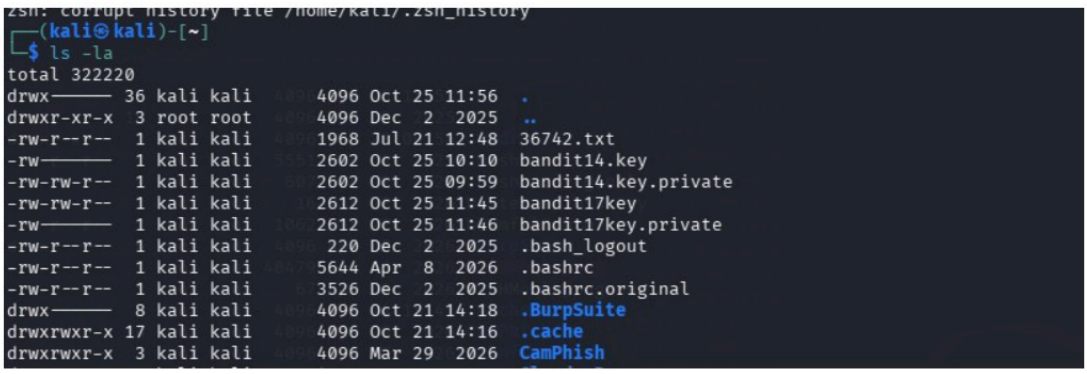


Figure 31: ls -la of my home directory showing the key files I had created.

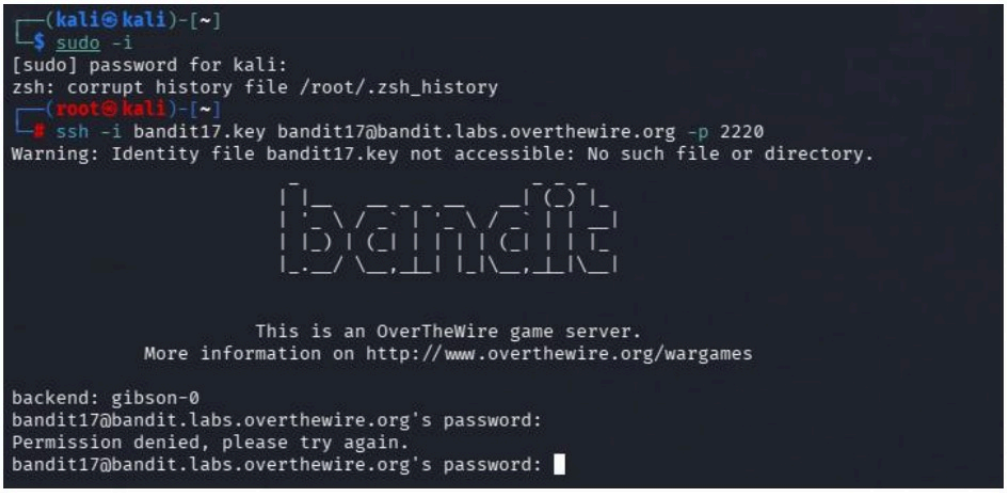


Figure 32: Trying sudo -i and ssh -i as root; still Permission denied.



Figure 33: The fix: file and head reveal the extra "Correct!" line; sed cleans it, ssh-keygen -y verifies it and ssh -i logs in.

There is no new password for this level; the private key bandit17_clean.key takes its place for logging in as bandit17.

Level 17 → Level 18

Objective

Two files are in the home directory: passwords.old and passwords.new. The password for the next level is the only line that was changed between them.

Commands Used

```
diff passwords.old passwords.new
```

Reasoning

diff compares two files line by line. In its output, the line that begins with "<" is from the first file (the old one) and the line beginning with ">" is from the second file (the new one). The task says the password is in passwords.new, so the line marked with ">" is the answer, and the "<" line is the old, obsolete value. Mixing these up would have been an easy mistake.

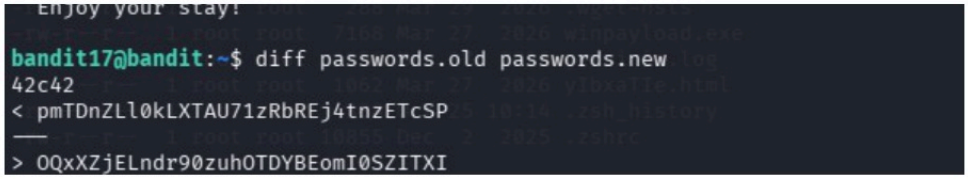


Figure 34: diff passwords.old passwords.new, the ">" line is the password.

> **Password for Level 18: `OQxXZjELndr90zuhOTDYBEomI0SZITXI`**

Level 18 → Level 19

Objective

The .bashrc file of bandit18 was modified so that you are logged out immediately when you connect. The password for the next level is in a file called readme in the home directory.

Commands Used

```
ssh bandit18@bandit.labs.overthewire.org -p 2220 cat readme
```

Reasoning

An interactive login runs .bashrc, which here contains a command that closes the session. But SSH can also run a single command on the server instead of opening a shell, and the server still runs it for me. By putting cat readme at the end of the ssh command, I read the file and left again before the logout trap could stop me. My first try gave "Permission denied, please try again" because I mistyped the password, and the second attempt worked.

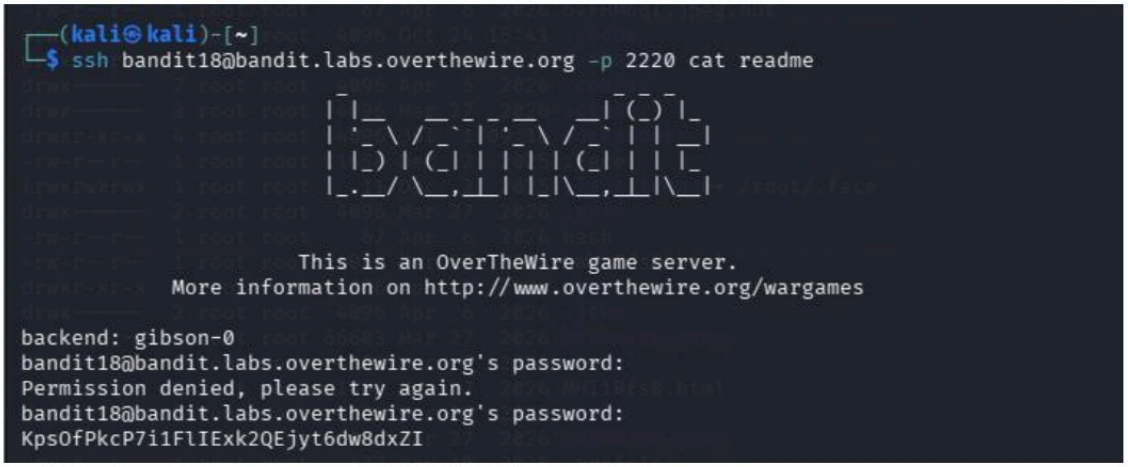


Figure 35: Running cat readme remotely through ssh, bypassing the modified .bashrc.

> **Password for Level 19: `KpsOfPkcP7i1FlIExk2QEjyt6dw8dxZI`**

Level 19 → Level 20

Objective

To get the password for bandit20, use the setuid binary in the home directory (bandit20-do) which executes commands as another user.

Commands Used

```
ls
ls -l
./bandit20-do cat /etc/bandit_pass/bandit20
```

Reasoning

ls -l showed -rwsr-x--- owned by bandit20 with the s in the permissions. That "s" is the setuid bit: when someone runs the program, it executes with the rights of the file owner, in this case bandit20. That is why the program can read /etc/bandit_pass/bandit20, which bandit19 could never read directly. My first attempt used the wrong path (/etc/bandit_pass/bandit20-do) and returned "No such file", and the corrected path returned the password. Setuid is powerful and dangerous, because any flaw in such a program gives an attacker the owner's privileges.

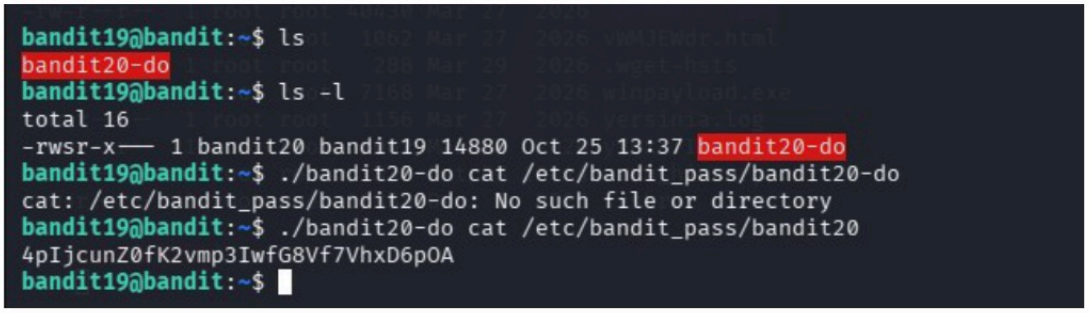


Figure 36: Running the setuid helper bandit20-do to read the next password.

> **Password for Level 20: `4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA`**

Level 20: Login Reached

Objective

Use the password collected in Level 19 to log in as bandit20, completing the chain of levels 0 to 20.

Commands Used

```
ssh bandit20@bandit.labs.overthewire.org -p 2220
```

Reasoning

The login worked on the first attempt and the server printed "Welcome to OverTheWire!", which confirmed that every password in the chain was correct and that I had reached Level 20.



Figure 37: Successful login as bandit20.

6. Summary of All Levels

Level	Concept	Main command(s)	Password / result
0 → 1	SSH basics	ssh -p 2220, cat readme	`6y2kwnwK6grgvwvpvLaa2T1cpFEKOhNR`
1 → 2	Dash filename	cat ./-	`PK8fYLZg2hnHSz83plBL1iEPKdD3QToB`
2 → 3	Spaces and dashes	cat < "--spaces in this filename--"	`7ZZ2LFrykP2zEyvBl4m3clcL7tGYJPME`

Level	Concept	Main command(s)	Password / result
3 → 4	Hidden files	ls -a, cat .Hiding-From-You	`Read from hidden file`
4 → 5	Identify file types	file ./*, cat ./-file07	`6C7h9GD8M6ai5nr7wo1RonrzFjj9yIrG`
5 → 6	Search by size	du -b -a | grep 1033	`pXa26xhMWaC2SvDotA4r9EgZkulOeSBW`
6 → 7	find by user, group, size	find / -user -group -size 33c	`Bmnnvf82KzQlfxgAI2d1zYbr1u9pr3E3`
7 → 8	grep	grep millionth data.txt	`VR1ljMayciFxbnUokuQmJFw6QC9VKtub`
8 → 9	Unique line	sort | uniq -u	`EjmOSvuAu7sGAHqHVcBDPirRe9T03kxl`
9 → 10	Strings in binary	strings | grep "=+"	`B0s2khmbT9u0geKuOoVGW3JZKhndE3BG`
10 → 11	Base64	base64 --decode	`pYfOY6HwUsDj5rL9UvyhU7MCmv8vN5Ro`
11 → 12	ROT13	tr 'A-Za-z' 'N-ZA-Mn-za-m'	`GROozWPO8QyN0mGrjUkID0WCYkZiQxrN`
12 → 13	Hexdump and nested compression	xxd -r, gzip, bzip2, tar	`qQYQiHOBPR8zR61qxYqX45quvihF2uzk`
13 → 14	SSH private key	ssh -i key (chmod 600)	`aaWecNkG4FhxJQxz07uiwzVP6bJiYS65`
14 → 15	Netcat to a port	nc localhost 30000	`pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7`
15 → 16	SSL/TLS client	openssl s_client -connect :30001	`kS0Hf0u5HiXFwKMKFqXvPdOTNGGa0X8V`
16 → 17	Port scan and key	nmap -sV, openssl, sed	`Private key (bandit17_clean.key)`
17 → 18	File comparison	diff passwords.old passwords.new	`OQxXZjELndr90zuhOTDYBEomI0SZITXI`
18 → 19	Remote command over SSH	ssh ... cat readme	`KpsOfPkcP7i1FlIExk2QEjyt6dw8dxZI`
19 → 20	Setuid binary	./bandit20-do cat ...	`4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6pOA`

7. Challenges and Troubleshooting

- **Filenames that look like options (Levels 1, 2, 4):** cat - and cat -file07 do not work because the shell and the program treat the dash specially. Using ./ or redirection with < solved it.
- **Permission errors (Levels 6, 13):** many "Permission denied" lines clutter the find output, so 2>/dev/null helped. In Level 13 I learned that the home directory is not writable on the Bandit server.
- **Repeated decompression (Level 12):** I ran gzip -d twice on the same file and also lost track of the current file name. Checking with xxd or file after each step fixed this.
- **SSH keys (Levels 13 and 16):** a key with wrong permissions or an extra text line is rejected. chmod 600, head -n 2 and ssh-keygen -y were the tools that showed me the real problem.
- **Copy and paste mistakes:** a few "Permission denied, please try again" messages and the "Wrong!" answer in Level 14 were all caused by pasting an old or incorrect password.

8. Key Learnings

- Linux basics: navigation, hidden files, quoting and special characters in filenames, permissions and ownership.
- Text processing with grep, sort, uniq, strings, tr, diff, head and pipes, which make a big file easy to search.
- Encoding versus encryption: Base64 and ROT13 only hide data from casual reading and are trivially reversible.

- Working with archives: recognising gzip, bzip2 and tar by their headers instead of trusting file extensions.
- SSH authentication with passwords and private keys, why chmod 600 matters, and running remote commands over SSH.
- Basic networking: talking to a port with nc, scanning with nmap -sV, TLS connections with openssl s_client and recognising self-signed certificates.
- Privilege concepts: how the setuid bit works and why it is a sensitive feature.

9. Security Observations

Although Bandit is a game, many levels mirror real mistakes that administrators and developers make:

- Secrets stored in plain-text files, readable files in home directories, or hidden files (Levels 0, 3, 6) are found quickly by anyone with shell access.
- Passwords or keys left in backups or old files (Level 17) can leak data even after the credential was changed.
- Weak obfuscation such as Base64 or ROT13 (Levels 10 and 11) is not a security control.
- Self-signed certificates (Levels 15 and 16) do not allow a client to verify the server's identity, and clients must not blindly ignore such warnings in real environments.
- Setuid programs (Level 19) and shell start-up files that restrict users (Level 18) can often be bypassed if the design is not careful.
- Services listening on unexpected ports (Level 16) can be found in seconds with nmap, so unused ports should be closed.

10. Summary

In this practical I logged in to the OverTheWire Bandit server and solved the levels from Level 0 to Level 19, which gave me access to Level 20. For each level I identified the clue, chose a suitable command, verified the output, and collected the password for the next user. The path covered basic file handling, text searching and filtering, encoding and compression, SSH key authentication, port scanning, TLS connections and setuid binaries.

The levels that taught me the most were Level 12 (nested compression), Level 13 and 16 (SSH key handling) and Level 19 (setuid). In them, my first attempts failed, and finding out why they failed gave me more understanding than a quick success would have. Overall, the challenge improved both my command-line speed and my way of thinking: always inspect first, read the error message carefully, and change only one thing at a time.

11. Suggestions / Recommendations

- Never store passwords or private keys in plain text in home directories, scripts or shared folders; use a password manager or a secrets vault, and restrict file permissions (chmod 600 for keys).
- Prefer SSH key authentication with a passphrase over passwords, and disable password logins on real servers.
- Use trusted certificates and encrypted protocols (TLS) for every service; avoid plain netcat-style services for sensitive data.
- Do not rely on Base64, ROT13 or simple compression to protect data; use real encryption such as AES with proper key management.
- Keep setuid binaries to a minimum, review them regularly and audit who can execute them.

- Close unused ports and run regular scans (for example nmap) on your own systems to know what is exposed.
- Remove old backup and credential files after a password change, and keep the system logs for investigation.
- For learners: keep a notes file with every command and the reason for it, try an alternative solution after each level (for example find instead of du), and respect the OverTheWire rules by not publishing passwords publicly.

12. References

- OverTheWire Bandit: https://overthewire.org/wargames/bandit/
- OverTheWire rules: https://overthewire.org/rules/
- Linux manual pages used: man ssh, man find, man grep, man uniq, man tr, man xxd, man openssl-s_client, man nmap.